VDR-LLM-Prolog Integer GPU Compute Stack: TensorProlog

A GPU Compute Architecture for Exact Integer Inference, Knowledge Operations, and Autonomous Session Management

Registry: [HOWL-VDR-35-2026]

Series Path: [HOWL-VDR-1-2026] → [HOWL-VDR-2-2026] → [HOWL-MATH-3-2026] → [HOWL-MATH-4-2026] → … → [HOWL-VDR-14-2026] → … → [HOWL-VDR-21-2026] → [HOWL-VDR-22-2026] → [HOWL-VDR-23-2026] → [HOWL-VDR-24-2026] → [HOWL-VDR-25-2026] → [HOWL-VDR-26-2026] → [HOWL-VDR-27-2026] → [HOWL-VDR-28-2026] → [HOWL-VDR-29-2026] → [HOWL-VDR-30-2026] → [HOWL-VDR-31-2026] → [HOWL-VDR-32-2026] → [HOWL-VDR-33-2026] → [HOWL-VDR-35-2026]

DOI: 10.5281/zenodo.20289699

Date: May 2026

Domain: ML Infrastructure / GPU Architecture / Integer Compute

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Opus 4.6.

Abstract

Current GPU compute stacks route integer inference through hardware designed for floating-point arithmetic, then compensate with conversion layers, precision negotiation, NaN handling, loss scaling, and rounding mode management. VDR-LLM-Prolog eliminates floating-point computation entirely — the model's forward pass, attention mechanism, softmax, training loop, and all infrastructure operations run on exact integer arithmetic with fixed denominators. TensorProlog is the GPU compute layer built for this architecture. It defines: a type system with three fixed-denominator Q-bases (Q16, Q32, Q335) and no float types; an instruction set where multiply-accumulate is a widening integer multiply plus bit shift; a memory model organized around fixed-size knowledge base structs at integer addresses; session-scoped streams with integer credential enforcement; a Prolog engine parallelized as batched cross-multiply unification across warps; grammar-directed structural token generation that bypasses the LLM forward pass; a runner system that provides autonomous background execution with snapshot-based recycling; and a server layer that clones sessions per connection with time-bounded integer credentials. The spec covers 23 modules, approximately 580 API functions (versus ~4,000+ in CUDA), approximately 30,000 lines of implementation across 168 files, and builds in six phases from pure-Zig CPU arithmetic through GCP GPU deployment. The entire float software stack — cuBLAS precision variants, cuDNN mixed-precision management,

TensorRT quantization calibration, NCCL non-deterministic allreduce, loss scaling, gradient clipping, NaN recovery, and the Transformer Engine — is replaced, not optimized.

---

1. The Problem

A GPU streaming multiprocessor contains integer ALUs, floating-point ALUs, tensor cores that handle multiple float formats, and special function units for transcendentals. The software stack that drives this hardware — CUDA, cuBLAS, cuDNN, cuFFT, cuSOLVER, TensorRT, NCCL — manages the interactions between these unit types. A single matrix multiply may negotiate between FP16 inputs, FP32 accumulators, TF32 tensor core operations, and FP8 output quantization. The Transformer Engine dynamically selects between FP8 and FP16 per layer. Mixed-precision training requires loss scaling to prevent gradient underflow, gradient clipping to prevent overflow, warmup schedules to avoid early-training float instability, and NaN detection and recovery for when these mitigations fail.

This complexity exists because floating-point arithmetic has failure modes. NaN propagates silently through computation. Infinity results from overflow. Subnormals lose precision without warning. Rounding is platform-dependent when thread scheduling varies. Addition is non-associative, so distributed allreduce produces different sums depending on reduction tree topology. Two training runs on the same hardware with the same data produce different models because float accumulation order varies between runs.

Integer arithmetic has none of these failure modes. Integers cannot be NaN. Integers cannot be infinity. Integers have no subnormals. Integer addition is associative regardless of evaluation order. Integer arithmetic is deterministic across all platforms, all thread schedules, all reduction topologies. The result of an integer multiply on a GPU in Tokyo is identical to the result on a CPU in Dublin.

VDR arithmetic fixes the denominator at creation, represents every value as three integers (Value, Denominator, Remainder), and pushes overflow into an inspectable remainder slot rather than discarding it silently. When the denominator is a power of two, the divmod operation that separates quotient from remainder is a bit shift — zero logic gates in hardware, a single instruction on CPU, fixed wiring on an ASIC. The Q16 multiply-accumulate sequence (widening multiply, accumulate, right-shift by 16) is instruction-identical to INT8/INT16 quantized inference on existing tensor cores.

TensorProlog is the compute layer that results from removing float arithmetic from the GPU programming model and replacing it with VDR integer operations, knowledge base addressing, Prolog unification, grammar-directed generation, session lifecycle management, and autonomous runner execution.

---

2. VDR Arithmetic on GPU

2.1 The Type System

TensorProlog provides three fixed-denominator VDR types. No float types exist anywhere in the system.

Q16 is the primary operational type. D = 65536 = 2^16. The struct is 8 bytes: a 32-bit integer value, a 16-bit integer remainder, and 16 bits of padding for alignment. Precision floor is $1/65536 \approx 1.53 \times 10^{-5}$. This is the inference type — model weights, activations, attention scores, gradients, and optimizer state all live in Q16.

Q32 is the intermediate precision type. D = 2^32. The struct is 16 bytes: a 64-bit value, a 32-bit remainder at depth 0, and a 32-bit remainder at depth 1. Used for accumulation across long sequences where Q16 remainder builds up, and for representing values greater than 1.0 at full precision (Q16 can represent values > 1.0 via the i32 numerator, but the fractional precision remains 1/65536).

Q335 is the high-precision type for transcendental computation. D = 2^335. The struct is 240 bytes: six 64-bit limbs for the value (384-bit integer), plus four remainder levels at the same width. The remainder depth is fixed at 4. This is the engineering choice the paper series identifies: measure your workload, pick your depth, fix the struct. Depth 4 means anything beyond R3 is truncated, and R3 tells you exactly how much precision you left on the table. This is not rounding error — it is a declared, bounded, inspectable truncation at a known depth.

The denominator is never stored. It is a compile-time constant. The compiler tracks Q-basis per register and per pointer. Assigning a Q16 value to a Q32 register is a compile error. Converting between bases requires an explicit `reproject` call that computes exact remainders. There are no implicit conversions, no silent widening, no precision mode flags.

2.2 The Core Instruction

Every numerical operation in the system reduces to one instruction pattern:

Widening multiply: i16 $\times$ i16 $\rightarrow$ i32 (for Q16). Shift right by 16: quotient becomes the new value. Mask low 16 bits: remainder becomes R0.

On existing GPU tensor cores, this is the INT8/INT16 multiply-accumulate path. H100 delivers 3,958 TOPS on INT8 tensor cores versus 1,979 TFLOPS on FP16 — the integer path is 2 $\times$ the float path on the same silicon, because integer multiplication is simpler than float multiplication (no exponent handling, no mantissa alignment, no normalization, no rounding logic).

What disappears when you use only this path: the entire float pipeline. No FP4/FP6/FP8/FP16/BF16/TF32/FP32/FP64 type hierarchy. No conversion functions between these types. No mixed-precision orchestration. No Transformer Engine switching between FP8 and FP16 per layer. No special function units for transcendentals

(the quadratic softmax surrogate uses integer multiply and integer divide; exact transcendentals via FRU use iterative integer recurrence). No NaN/Inf/subnormal branches, which means no warp divergence from data-dependent special cases, which means full warp utilization on every cycle.

2.3 Softmax

Conventional softmax computes $\exp(x_i) / \Sigma\exp(x_j)$ using the GPU's special function units. SFUs are shared across warps and serialize — they are a throughput bottleneck on attention-heavy workloads.

VDR uses a quadratic surrogate: shift all logits so the minimum is zero, square each shifted value, divide each by the sum of all squares. The output is exact VDR fractions that sum to D by construction. Not approximately D. Exactly D. The toy LLM implementation confirms this: softmax_sum = 65536 on every epoch, every forward pass, every generation step, verified by integer equality.

The surrogate uses only integer multiply, integer add, and integer divide — full warp throughput, no SFU, no data-dependent divergence. Whether the quadratic gradient landscape matches exp-softmax at production scale is an open empirical question. The FRU (Functional Remainder Unit) on dedicated hardware makes exact exp-softmax available at competitive latency, so both paths are available as a configuration choice, not an architectural constraint [HOWL-VDR-23-2026].

2.4 Validation

The VDR arithmetic foundation has been validated across 884 tests in 37 domains (23 mathematical, 14 physical) with zero VDR computation errors. The Zig Q16 implementation achieves 688 nanoseconds per forward pass and 1.42 million tokens per second on a 2019 laptop with scalar CPU instructions, zero floating-point operations, zero heap allocations, and 2,368 bytes of total model memory. Loss monotonically decreases across training epochs. Generation is deterministic: same input produces same output on every run. These are measured values from running code, not projections.

---

3. Knowledge Bases on GPU

3.1 The Data Problem

Conventional LLM systems put data in the context window. A 1 KB JSON response from an API becomes ~320 tokens in the attention buffer. Every subsequent turn re-reads those tokens through the quadratic attention mechanism. At turn 20, the cumulative attention cost from re-reading prior data dwarfs the cost of the actual computation the LLM is performing. Longer context windows exacerbate this: a 128K context window means the LLM can hold more data, but at quadratic cost per turn to re-attend to all of it.

VDR-LLM-Prolog stores data in knowledge bases at integer addresses. A KB is a fixed-size struct (256 bytes, padded for cache alignment) containing 26 fields: identity (name,

path, sequential integer ID), persistent state (facts, rules, constraints, connections, grammars), live state (working data, LRU caches, counters, locks, queues, stacks, ring buffers, bitsets), structural links (parent ID, children IDs, mounts), and metadata (visibility level, frozen flag, owner, timestamps).

Accessing a fact is two integer operations: kb_id to locate the KB struct, slot_id to index into the fact store. O(1). The data never enters the token stream. The LLM emits an 8-token command referencing a dotted path, the system resolves the path to an integer via a hash map, and the compiled builtin operates directly on the data at that address. The 1 KB JSON file is parsed by a compiled parser, stored as KB facts, and queried via 8-token commands — at turn 1 or turn 1,000, the cost is identical [HOWL-VDR-5-2026].

3.2 GPU Memory Layout

KB structs occupy a contiguous region of GPU global memory. Fixed struct size means zero fragmentation and predictable cache behavior. A reference deployment with 100,000 KBs, 10 million facts, 100,000 rules, and 10,000 sessions requires approximately 2.2 GB of device memory for the entire infrastructure — negligible relative to model weights (56 GB for a 7B-parameter Q16 model). The infrastructure overhead is smaller than most float precision-management bookkeeping.

Shared memory on each SM serves as a KB cache. 16 KB structs and 512 facts fit in approximately 24 KB of shared memory, well within the H100's 228 KB per SM. A Prolog query loads candidate facts into this cache, runs cross-multiply unification in parallel across warps, and writes results back. The access pattern is small random reads by integer ID, not large sequential tile loads — the opposite of conventional weight-matrix access. This means the memory controller should optimize for latency rather than bandwidth when serving KB operations, a scheduling hint that TensorProlog exposes through distinct kernel launch types (MAC kernels for matrix work, Prolog kernels for KB-heavy work).

3.3 Bounded Data Primitives

Seven data primitive types provide working memory within KBs: LRU caches (1–1,000 entries, oldest evicted on overflow), counters (i32 range, clamp at declared min/max, never wrap), locks (non-blocking coordination signals), queues (bounded FIFO, 1–1,000 entries, push returns false when full), stacks (bounded LIFO, same constraints), ring buffers (fixed-size, oldest overwritten), and bitsets (1–10,000 bits, fixed at creation).

No primitive can grow beyond its declared bound. Memory footprint per session is fixed at creation and cannot increase. This is why sessions with thousands of active KBs and thousands of data primitives maintain constant memory footprint indefinitely — the invariant is structural, not managed by garbage collection or memory pressure heuristics.

---

4. Prolog on GPU

4.1 Why Prolog

The LLM generates reasoning as prose — "if the error rate exceeds 15% and a recent deploy changed the connection pool size, then the root cause is likely pool undersizing." This costs 50–200 tokens, exercises a full forward pass per token, and is discarded when the session ends. The next time the same pattern appears, the LLM generates the same reasoning from scratch.

A Prolog rule captures the same logic as a deterministic pattern: if error_rate_high(Service, Rate) and config_change(Service, max_pool_size, OldSize, NewSize) and NewSize < OldSize, then root_cause(pool_undersized, Service). The rule costs 25–40 tokens to formalize. It fires automatically on every future matching pattern at zero LLM cost. By investigation 100, 150+ such rules handle 93% of triage automatically.

4.2 GPU-Parallel Unification

Unification — determining whether two Prolog terms can be made identical by substituting variables — is the core operation. For VDR values, unification is cross-multiply comparison: a.v $\times$ b.D versus b.v $\times$ a.D. Since all values in a session share the same Q-basis, this simplifies to comparing integer numerators directly.

On GPU, fact matching is parallelized as follows: load candidate facts into shared memory (one SM's KB cache holds 512 facts). Distribute candidates across warps (32 threads per warp, each thread handles one candidate). Each thread performs the cross-multiply comparison for its candidate. Filter matches via warp-level vote. Collect surviving candidates for recursive body evaluation. This is structurally identical to a parallel database join where the comparison operator is integer equality rather than float distance.

Depth-first search with backtracking remains sequential within a branch, but independent branches can be explored in parallel across SMs. The depth limit is 100, matching the system's declared maximum recursion depth [HOWL-VDR-23-2026].

---

5. Grammar-Directed Structural Token Generation

5.1 The Token Waste

Measured structural token percentages by output type: Python code ~40%, JSON ~55%, formatted tables ~65%, English prose with data ~30%, compacted pipe-delimited tables ~80%. Every structural token — every brace, bracket, colon, comma, pipe, header, indentation character — is predicted through a full forward pass over 50,000+ vocabulary items with softmax normalization. The prediction is probabilistic. The LLM can and does produce malformed JSON, mismatched brackets, and broken table formatting.

5.2 Grammar Replacement

A grammar template declares typed slots for content: `"{name:text}: {value:vdr_value} ({confidence:vdr_value})"`. The LLM or KB fills the content slots. The grammar fills everything else. Compilation validates the template: all braces matched, all slot types valid, template structurally correct for any valid fill. Rendering walks the template, copies literal bytes, and renders fill values at slot positions.

Every structural byte comes from the template. Not from softmax prediction. Not from a forward pass. From a memcpy of literal bytes and an integer-to-string conversion for values. The grammar guarantees syntactic correctness by construction. JSON output cannot be malformed because the grammar produces every brace, bracket, colon, and comma.

Grammars persist in KBs. They inherit through the KB tree (a child KB inherits its parent's grammars). A grammar created during investigation 1 formats output for every subsequent investigation at zero cost per reuse [HOWL-VDR-12-2026].

---

6. Sessions, Snapshots, and Clones

6.1 Session Isolation

Each session has: a user_id (i32), a visibility level (enum), a KB root (i32), a set of grants, and bounded resource limits. The session's TensorProlog stream carries these credentials. Every kernel launched on that stream inherits them. Every KB access checks them — two integer comparisons per ancestor in the scope walk. Data that fails the check is absent from the session's view. Not filtered. Not redacted. Absent. The query path never touches the data.

6.2 Snapshots

A snapshot is an atomic capture of all session state: KB structs, facts, rules, terms, text, grammars, live state (all seven primitive types), and grants. The format is a contiguous binary blob with a header containing region sizes and a CRC32 checksum computed over integer data. The checksum is an integer — deterministic.

Restore overwrites session state from the snapshot after validating the checksum. After restore, the session is in exactly the state at snapshot time. Not approximately. The word "exactly" is meaningful here because the representation is integer. A float snapshot can be exactly restored in the sense that the same bits are loaded, but the first operation on those bits may produce a different result due to thread scheduling or rounding mode. An integer snapshot is restored and the first operation produces the same result. Always.

Typical snapshot size: 10 KB to 500 KB for operational sessions. Model weights are not in the snapshot — they are shared and loaded separately. The snapshot captures session state, not model state.

6.3 Clones

Cloning creates a new session that shares the parent's persistent KBs via copy-on-write. The clone's page table points to the parent's memory pages. On first write, the page is copied to the clone's private region. Subsequent writes go to the private copy. The parent never sees the clone's modifications. The clone never sees the parent's subsequent modifications.

Clone-from-snapshot is the operational primitive for freshness. A processor runner that has been ingesting data for 200 turns has accumulated KV-cache state and live working memory. Some of this state may have drifted from the session's intended behavior. The fix: snapshot the session (capturing all accumulated knowledge), kill the session (destroying all live state including any drift), clone from the snapshot (fresh session with identical knowledge). The paper's phrase is: "the snapshot is the factory. Clones are disposable. Knowledge persists. Drift dies."

This mechanism depends on exact reproduction. The clone must start from a state bit-identical to the snapshot. If the snapshot contained float values, the clone's first operation on those values could diverge from the original due to different thread scheduling on the new session's stream. With integers, the clone's first operation produces the same result as the original would have — the clone is a perfect fork. Over hundreds of recycle cycles across months of operation, each clone generation starts from exact state. There is no generational drift from copy-of-a-copy degradation because integer copying is exact [HOWL-VDR-8-2026].

---

7. Structural Safety

7.1 Access Control

Data access is gated by integer comparison before the LLM is involved. The session's integer user_id is set at authentication. The KB's integer visibility level is set by the KB owner. The access check walks from the target KB up the parent chain, comparing visibility at each ancestor. If any ancestor is unreachable, the target is invisible.

No prompt modifies any integer in any access control check. Role-play does not change the integer user_id. Many-shot jailbreaking does not modify visibility levels. Encoding tricks do not bypass integer comparison. The LLM is an untrusted component operating between pre-filtered input and post-validated output.

7.2 Grants

Operational primitives (filesystem read/write, script execution, network fetch, process management) require a positive credential grant with default denial. A grant is a struct: class (filesystem/compile/execute/lint/network/process), target pattern, remaining uses (i32, -1 for unlimited), expiry timestamp (i32, 0 for never), holder user_id. Grant check is: does the session's user_id hold an active grant matching the requested class and target? Active means: state == ACTIVE AND (expires_at == 0 OR current_time < expires_at)

AND (remaining_uses == -1 OR remaining_uses > 0). Four integer comparisons. If granted, remaining_uses decrements atomically.

No runner can modify its own grants. Grant modification requires admin-level grants that no operational runner holds. Revocation is permanent. All transitions are logged as audit entries with timestamps.

7.3 Audit

Every access check, grant check, fact assertion, fact retraction, rule firing, session creation, session destruction, and operational execution produces an audit entry: timestamp (i32), session_id (i32), user_id (i32), action (enum), target_kb_id, target_slot_id, grant_id, and result (allowed/denied). Entries are written to an append-only ring buffer in device memory. The ring has a declared capacity (default: 1,000,000 entries). When full, the oldest entry is overwritten.

Zero LLM tokens are spent on safety. The entire access control, grant enforcement, and audit trail operates on integer comparison and integer storage.

---

8. The Universal Execution Cycle

Every operation in the system — interactive chat, polling runner, processor ingest, batch task, HTTP request, WebSocket message — is an instantiation of one function.

8.1 Phase 0: Pre-LLM Rule Evaluation

Before the LLM sees anything, fire all matching Prolog rules against the current KB state. This is the L3 path. Zero tokens. Pure integer unification. If the rules fully handle the input (produce a finding with confidence above the auto-response threshold and an associated grammar for rendering), the cycle completes without invoking the LLM at all. The output is grammar-rendered from KB facts.

In a mature deployment, 93% of operations resolve here. The LLM is not invoked. The GPU's integer ALUs run Prolog unification, the grammar engine renders the response, and the LLM's forward pass is never scheduled.

8.2 Phase 1: Context Assembly

If Phase 0 did not fully resolve the input, build the LLM's context. The context contains: a system prompt from the seed KB (~200 tokens, cached after first load), the active scope reference (~5 tokens), a scratchpad with results from auto-fired rules (~0–50 tokens), and the current turn's input. The context does not contain prior turns (they are in KBs), data (it is at KB addresses), prior reasoning (it is in Prolog rules), or formatting templates (they are in grammars).

Context size is approximately constant regardless of turn number. Turn 1: ~350 tokens. Turn 1,000: ~350 tokens. The LLM's attention window does not grow with conversation length.

8.3 Phase 2: LLM Generation + Command Dispatch

The LLM reads the context and generates a response — a mix of command tokens (dispatched to the system), prose tokens (passed through to output), and direct-output references (resolved from KB).

Command tokens use a constrained vocabulary of ~300 known operation names. Each command is approximately 8 tokens at ~2 bits of entropy per token (selecting from a small known set). Reliability is ~99.2% per command, versus ~86% for conventional JSON function calling which generates every brace and parameter name through full-vocabulary softmax.

When a command token is detected, the cycle switches to constrained generation (vocabulary masked to command names only), parses the command into a typed struct, runs the access check (integer comparison), runs the grant check if operational (integer comparison), and dispatches execution. Results go to the scratchpad, not to the output. The LLM can inspect results on its next generation step.

When a direct-output reference is detected (a `kb://` URI), the cycle loads the referenced data from KB, finds the associated grammar by walking the KB tree, and renders the data through the grammar into the output stream. The data traveled from KB integer address to formatted output without the LLM generating any of its structural characters.

8.4 Phase 3: Post-Cycle

Update session counters (turn number, tokens consumed, commands executed). Update level statistics (L1/L2/L3 distribution as exact fractions). Auto-snapshot if configured. Check turn budget for runners that need recycling.

8.5 Execution Levels

L1 — Full LLM judgment. 50–500 tokens. No stored rule covers the situation. The LLM exercises full reasoning. At the end, it may formalize its judgment as a Prolog rule (25–40 tokens), transitioning this pattern from L1 to L2 for future encounters.

L2 — LLM invokes stored rule. ~8 tokens. The LLM recognizes that a stored rule applies, emits a query command, and wraps the result in prose framing. Total cost: ~18 tokens.

L3 — Automatic rule firing. 0 tokens. The Prolog rule fires during Phase 0 without LLM involvement. The cycle resolves from grammar-rendered KB data.

The accumulation curve from the paper: investigation 1 costs 329 command tokens with 0 auto-firing rules. Investigation 10: 92 tokens, 65% auto-triage. Investigation 100: 55

tokens, 93% auto-triage. The system gets cheaper and more capable simultaneously because solved problems stay solved as persistent Prolog rules at integer addresses [HOWL-VDR-19-2026].

---

9. Runners

9.1 The Autonomy Problem

A conventional LLM exists only when invoked. Between requests, nothing happens. The LLM has no persistent state, no background processing, no ability to detect conditions and act without human prompting.

Runners provide autonomous execution. Each runner type is a loop around the universal cycle with a different input source and output destination.

9.2 Poller

A poller executes the universal cycle on a timer (configurable interval, default 60 seconds). Input is synthetic ("poll cycle N"). Phase 0 fires all triage rules against current KB state. If rules handle everything, zero LLM tokens are consumed. Output goes to a notification KB or external webhook.

Each poll cycle gets a fresh TensorProlog stream. The LLM's KV-cache is not carried between cycles. No attention degradation accumulates across polls.

9.3 Processor

A processor maintains a persistent connection to an external data source (Prometheus, deploy pipeline, alerting system, custom API). Each incoming data item triggers a mini-cycle that compacts the data into KB facts. Known patterns are handled by Prolog rules (L3). Novel patterns fall through to the LLM (L1), which can formalize new rules for future encounters.

At a configured turn threshold (default 200), the processor recycles: snapshot the session, kill it, clone from the snapshot, restore the external connection from saved state. The data stream is continuous. The LLM processing it is always fresh. Knowledge persists through the snapshot. Drift dies with the killed session.

Reconnection uses exponential backoff: 1s, 2s, 4s, 8s, 16s, 32s, 60s cap. Connection state (socket descriptors, protocol buffers, sequence numbers) is saved before recycle and restored after.

9.4 Internal

An internal runner executes a configured computation function on a schedule. No external connections. Typical work: compute rolling averages as exact VDR fractions, detect trends via exact integer comparison, identify coverage gaps in the rule base, aggregate

metrics. All computation is KB queries plus exact VDR arithmetic plus KB assertions. Zero LLM tokens unless a genuinely novel pattern appears.

9.5 Batch

A batch runner pulls tasks from a KB queue, clones the parent session for isolation, processes each task in the clone, merges results back to the parent, and kills the clone. Up to max_concurrent clones active simultaneously. Clone-per-task provides isolation: if a task corrupts working state, only its clone is affected. The parent session and all other tasks are untouched.

9.6 Thread Pool

All runners are scheduled on a host-side thread pool. Thread count defaults to half of available hardware threads. Each runner iteration is submitted as a task. The pool manages work distribution. Runner state (RUNNING, STOPPED, ERROR, RECYCLING) is tracked in a runner table.

---

10. Server Architecture

10.1 Connection Handling

The server accepts TCP connections, performs protocol-specific handshake (HTTP, WebSocket, SMTP, MQTT, or raw TCP), authenticates the client, and creates a session clone from a template snapshot.

The template snapshot is the factory. It contains the KB tree with domain-specific rules, grammars, and seed data. Every incoming connection starts from a bit-identical state, then diverges based on the connection's specific interactions. The template is created once (either loaded from a file or built by configuring a session and snapshotting it), and cloned per connection.

10.2 Credential Lifecycle

Authentication extracts a token from the protocol handshake (HTTP Authorization header, MQTT username/password, SMTP AUTH command). The token is hashed (deterministic integer hash) and looked up in an auth KB by hash value. If found: the user's visibility level, grants, and account status are loaded from KB facts at integer addresses.

A credential is issued: user_id (i32), visibility_level (i8), a set of pre-resolved grants, issued_at timestamp (i32), and expires_at timestamp (i32). The credential is valid as long as: the valid flag is true AND the current timestamp is less than expires_at. Two integer comparisons. Checked before every request in the connection's message loop.

When the credential expires, the server sends a protocol-appropriate response (HTTP 401, WebSocket close frame with code 4001, SMTP 535) and the connection enters

the draining state. There is no "approximately expired" — the timestamp comparison is integer.

10.3 Request Processing

Each request is transformed into a vlp_input and passed to the universal cycle. The cycle runs on the connection's session-bound TensorProlog stream. The output is transformed into a protocol response via grammar templates.

Protocol grammars handle wire format. An HTTP response consists of: a status line grammar (`HTTP/1.1 {status_code:integer} {reason:text}`), header grammars (`{name:text}: {value:text}`), and a body grammar appropriate to the content type. Every structural byte of the HTTP response — every colon after a header name, every , every brace in JSON — comes from the grammar. The LLM generates content that fills grammar slots. The grammar generates everything else.

For stateless protocols (HTTP without keepalive), the session clone is destroyed after the response. For stateful protocols (WebSocket, SMTP sessions), the session persists across messages within the connection, accumulating KB state and rules. Snapshot on disconnect preserves the session for future restoration.

10.4 Rate Limiting

Request rate is tracked via bounded counters in the auth KB. One counter per user, reset at window boundaries. The limit check is: is the counter value >= max_requests? Integer comparison. Increment on allow. Reset on new window. No drift, no false threshold crossings, no approximate counting.

---

11. Confidence Propagation

11.1 Replacing Hedging Language

Conventional LLMs generate confidence as prose: "it appears that," "likely," "approximately." These communicate nothing quantifiable. VDR-LLM-Prolog replaces this with exact fractions from a declared confidence hierarchy.

The knowability spectrum assigns exact VDR fractions by source type: VDR computation = 1/1, Prolog derivation = 1/1, database query = 98/100, Prometheus metric = 95/100, script output = 95/100, REST API response = 85/100, peer-reviewed claim = 80/100, user-stated fact = 70/100, web search result = 50/100, LLM-generated content = 30/100, unknown = 0/1. These are stored as KB facts in the seed layer.

11.2 Propagation Formulas

Multiple agreeing sources combine as: $1 - \prod(1 - C_i)$. Two sources at 95/100 produce: $1 - (5/100)^2 = 9975/10000$. Conflicting sources degrade via a configurable penalty multiplier. A chain of N links at confidence C each produces C^N. All computed as exact VDR

fractions using integer multiply and integer divide. Not generated as hedging language. Not approximated. Computed.

Provenance tracking follows the derivation chain: if a fact was derived by a Prolog rule, its confidence is the chained product of its source facts' confidences. The chain is traceable: every fact has a provenance field recording source_type, source_kb_id, source_slot_id, confidence, timestamp, and derivation_rule_id. Follow the chain from any fact back to its original sources. Every step is exact.

---

12. TensorProlog API Surface

12.1 Module Structure

TensorProlog consists of 23 modules totaling approximately 580 functions.

Core runtime (36 functions): device management, memory allocation with Q-basis typing, session-scoped streams, event timing, kernel launch with scheduling hints (MAC/Prolog/Primitive).

VDR math (17 functions): GEMM (one function replacing cuBLAS's dozens of precision-variant entry points), batched and strided GEMM for multi-head attention, softmax (quadratic surrogate and exact exp via FRU), layer normalization, elementwise operations (add, sub, mul, div, dot, scale, compare), Q-basis reprojection, remainder normalization and monitoring.

Attention (3 functions): fused forward (QK^T → scale → mask → softmax → AV), backward with exact gradients, and softmax sum verification (expected: always zero violations, since exact sum = D is a structural invariant, not a tolerance check).

Training (10 functions): SGD, momentum, Adam (all in exact integer arithmetic), cross-entropy loss, backward pass via reverse-mode autodiff on an exact computation graph, checkpoint save/restore (bit-identical across platforms).

Knowledge bases (14 functions): KB create/destroy/freeze/unfreeze/info, fact assert/query/retract/search/scoped-search, child listing, mount/unmount, path resolution.

KB primitives (30 functions): create/get/put/clear for LRU, counter, lock, queue, stack, ring buffer, and bitset. Each with capacity enforcement.

Prolog (8 functions): rule assert/retract, query with scoped search, fire-all, fire-and-commit, low-level unification, rule statistics, hygiene scan (stale/failing/orphaned detection).

Grammar (7 functions): compile, render, render-from-KB, validate, slot listing, inherit, compose.

Runner (8 functions): create (poller/processor/internal/batch), start, stop, kill, status, recycle, destroy.

Session (10 functions): create, destroy, snapshot, restore, clone, merge, kill, info, snapshot-save-to-file, snapshot-load-from-file.

Safety (6 functions): access check, grant create/check/revoke/list, audit log query.

Confidence (5 functions): assign from source type, combine agreeing, combine conflicting, chain, propagate through derivation.

Distributed (10 functions): allreduce (deterministic — integer sum is associative), broadcast, allgather, reduce-scatter, communicator management, KB sync across devices, snapshot broadcast.

Transform (4 functions): exact DFT/IDFT (roundtrip returns original values exactly), 1D and 2D convolution.

Linear algebra (8 functions): matrix-vector multiply, transpose, Gaussian elimination, inverse, determinant, Gram-Schmidt, eigenvalues, SVD. All exact.

Statistics (8 functions): mean, variance, median, Bayesian update, normalize, histogram, correlation. All exact fractions.

Number theory (7 functions): GCD, LCM, modular exponentiation, Chinese Remainder Theorem, Euler's totient, primality, factorization.

Functional remainder (8 functions): sqrt, exp, log, sin, cos, atan2, batch resolve, FRU availability check.

Builtins (448 functions): the deterministic primitives from the VDR-LLM-Prolog specification, exposed as device-callable functions. 404 pure (no side effects, bounded termination, infallible on valid input). 44 operational (side effects, grant-gated).

Profiling (4 functions): start/stop, kernel stats, session stats, determinism verification (run N times, compare bit-by-bit; expected: always identical).

12.2 What's Eliminated

Approximately 3,400+ API functions from the CUDA ecosystem: cuBLAS precision variants, cuDNN layer functions at multiple precisions, cuFFT float transform plans, cuSOLVER float decompositions, cuSPARSE float sparse operations, TensorRT quantization calibration and precision selection, NCCL non-deterministic reduction operations, the entire Thrust and CUB template library for float types, and CUTLASS mixed-precision GEMM kernels.

The reduction is not from missing functionality. It is from the removal of precision-variant duplicates. cuBLAS GEMM has entry points for every combination of input type, output type, accumulator type, and compute type. TensorProlog GEMM has one entry point with a Q-basis parameter.

12.3 What's New

Approximately 90 functions across session, KB, Prolog, grammar, runner, safety, and confidence that enable capabilities with no equivalent in CUDA: persistent knowledge

at integer addresses, deterministic deduction, structural token generation, autonomous background execution, snapshot-based lifecycle management, integer access control, and exact confidence propagation.

---

13. Implementation

13.1 Language and Dependencies

The implementation is in Zig (0.14). External dependencies for Phases 1–4: zero. Pure Zig. Phase 5 adds the CUDA-like toolkit for GPU kernel compilation (Zig C interop for kernel launches, PTX inline assembly or CUDA C for kernel bodies). Phase 6 adds the GCP SDK for deployment automation.

The system has no dependency on PyTorch, TensorFlow, JAX, ONNX, or any ML framework. It is not a wrapper. It is not a plugin. It is the compute stack.

13.2 Build Phases

The system builds in six phases. Each phase produces a testable system. Each phase's tests remain valid for all subsequent phases. Nothing is rewritten — later phases add modules.

Phase 1: Arithmetic + KB Store. VDR Q16 type and operations, KB store with fact CRUD, tree relationships, path index, visibility checks, text store. ~2,500 lines. Testable locally on any machine, no GPU. Deliverable: the system can create KBs, assert facts, query facts, enforce visibility. All integer, all exact.

Phase 2: Prolog + Grammar + Session. Prolog engine (term representation, unification, depth-first query with backtracking, rule storage and firing, hygiene detection). Grammar engine (template compilation, slot-typed rendering, KB inheritance, composition). Session lifecycle (create, snapshot, restore, clone with COW, merge, kill). Seven bounded data primitive types. Grant system. Audit ring buffer. Confidence propagation. ~5,500 lines. Testable locally. Deliverable: full deduction, grammar rendering, session lifecycle, bounded working memory, structural safety. L1→L2→L3 transition is mechanically possible.

Phase 3: Inference Loop + Builtins. The universal execution cycle. Context builder. Command token parser and dispatcher. Auto-resolution check. LLM engine (model loading, forward pass, attention with exact softmax, autoregressive generation, KV-cache in KB, sampling). ~200 builtins across text, collections, sets, mappings, arithmetic, conversion, linear algebra, statistics, graph, integer/bit, and time categories. Seed layer initialization (engineering principles, hygiene rules, confidence table, command vocabulary). ~7,000 lines. Testable locally with the toy model (181 parameters, 5-token vocabulary). Deliverable: the full cycle runs end-to-end on CPU. The critical test: a scripted replay of the SRE investigation from the paper (Section 10.1), verifying token counts, KB state, rule creation, grammar rendering, and confidence values.

Phase 4: Runners + Server. Thread pool for runner scheduling. Four runner types (poller, processor with recycle, internal, batch with clone-per-task). HTTP and WebSocket server (accept loop, connection handler with clone-from-template, authentication via auth KB, credential lifecycle with integer expiry, rate limiting via bounded counters, health/metrics endpoint with grammar-rendered JSON, idle connection reaper, graceful shutdown with session snapshotting). Grant-gated operational builtins (filesystem, network, execute, compile, process). Configuration file parsing. ~6,000 lines. Testable locally. Deliverable: the system is a network-accessible daemon with autonomous background processing.

Phase 5: TensorProlog GPU Kernels. Device initialization and memory layout. GPU kernels: Q16 matrix multiply-accumulate, quadratic softmax, fused attention, exact layer normalization, elementwise operations, parallel Prolog unification, parallel sort. KB store on device memory. Host-device transfer. Profiling. ~6,000 lines (Zig + CUDA C). Requires GPU — first GCP phase. Deliverable: all operations from Phases 1–4 run on GPU with identical results (verified by cross-platform determinism tests comparing GPU output to CPU reference, byte-by-byte).

Phase 6: Production Integration. Multi-device model parallelism. Distributed deterministic allreduce. KB replication and sync. Load balancing. Monitoring (Prometheus-compatible metrics export). Full protocol implementations (SMTP, MQTT, DNS). ~3,000 lines. Requires multi-GPU GCP instances. Deliverable: production-scale deployment.

13.3 Line Counts

vdr/	5 files	~800 lines
kb/	7 files	~1,700 lines
prolog/	6 files	~1,500 lines
grammar/	5 files	~900 lines
session/	4 files	~1,200 lines
primitives/	8 files	~1,400 lines
safety/	3 files	~600 lines
confidence/	2 files	~300 lines
engine/	8 files	~2,000 lines
llm/	7 files	~1,800 lines

```
builtins/       12 files     ~4,000 lines

seed/            5 files       ~800 lines

runner/          6 files     ~2,000 lines

server/          8 files     ~2,500 lines

protocol/        5 files     ~1,500 lines

ops/             5 files     ~1,000 lines

config/          3 files       ~500 lines

gpu/            11 files     ~6,000 lines

deploy/          5 files     ~1,500 lines

test/          ~50 files     ~5,000 lines

Total:      ~168 files   ~30,000 lines code + ~5,000 lines tests
```

The paper series estimated ~20,500 lines across 65 modules for the VDR-LLM-Prolog system. The additional ~10,000 lines cover: TensorProlog GPU kernel implementations, server infrastructure (HTTP/WebSocket handling, connection management, TLS), protocol grammars, GCP deployment integration, and comprehensive test coverage. These are the components that bridge between the paper's architectural specification and a running system on real hardware.

---

14. Testing

14.1 Test Strategy

Every operation is tested for correctness, and every operation is tested for determinism. These are different properties. Correctness means the output is the right value. Determinism means the output is the same value every time, on every platform. In float systems, you can have correctness-within-tolerance without determinism (two runs produce different but both "correct" results). In TensorProlog, correctness and determinism are the same test: if the output is wrong, the determinism test fails, because the wrong output differs from the right output that was produced last time.

14.2 Test Phases

Phase 1 tests (~119 tests): Q16 arithmetic, KB CRUD, fact operations, tree traversal, path resolution, visibility checks. All local, no GPU.

Phase 2 tests (~151 additional, ~270 total): Prolog unification, queries, rule firing, hygiene detection, grammar compilation and rendering, session lifecycle (snapshot roundtrip, clone independence, merge conflict detection), all seven data primitive types (bounds enforcement on every primitive), grant checks, audit entries, confidence propagation.

Phase 3 tests (~207 additional, ~477 total): end-to-end cycle with toy model, context assembly, command parsing and execution, auto-resolution, forward pass correctness (softmax sum = D every time), deterministic generation (1,000 runs, identical output), ~180 builtin tests, seed layer initialization, and the SRE scenario replay.

Phase 4 tests (~55 additional, ~532 total): HTTP and WebSocket request handling, authentication, rate limiting, credential expiry, graceful shutdown, runner lifecycle (poller fires rules, processor recycles, batch clone-per-task), filesystem and network operational builtins with grant enforcement, full integration test (server + runners + client).

Phase 5 tests (~60 additional, ~592 total): GPU arithmetic correctness (matches CPU output byte-by-byte), GPU softmax (sum = D on GPU), GPU attention (weights match CPU), GPU determinism (100 runs, all identical), GPU forward pass (toy model matches CPU bit-for-bit), GPU parallel Prolog (matches sequential), cross-platform determinism (GPU output identical to CPU reference).

Phase 6 tests (~20 additional, ~612 total): multi-GPU model parallelism, distributed allreduce determinism, load testing, chaos testing (kill instances, verify recovery from snapshots).

14.3 Sustained Operation

A 24-hour test with simulated SRE workload validates the accumulation curve. Simulated Prometheus data generates metric patterns: 70% known patterns (should be L3 by hour 2), 20% variants (L2 after first encounter), 10% novel (L1). After 24 hours, the health endpoint reports the L1/L2/L3 distribution as exact fractions. Expected: L3 > 80%. Rules should have grown from the seed 15 to 50–80. Errors should be zero (deterministic system, no float instability). Replaying the same 24-hour sequence should produce byte-identical output.

14.4 Invariants

Ten invariants hold at all times, in all states, on all devices. Violation of any invariant is a system bug.

1. Softmax outputs sum to D exactly.
2. KB facts at integer addresses are exact and do not degrade.
3. Bounded primitives cannot exceed declared bounds.
4. Snapshot restore is bit-identical to snapshot state.
5. Clone COW is invisible to parent.

6. Access-denied data is absent, not filtered.
7. Grant denial happens before execution.
8. Integer arithmetic is deterministic across devices.
9. Prolog unification uses exact comparison.
10. Audit log is append-only and complete.

---

15. GCP Deployment

15.1 Instance Types

Phase 5 development and testing: n1-standard-8 with 1 × NVIDIA T4 (~$0.35/hour). T4 provides INT8 tensor core throughput sufficient for kernel validation and correctness testing against CPU reference.

Phase 5 benchmarking: a3-highgpu-1g with 1 × H100 80GB (~$12–15/hour). H100 provides 3,958 TOPS INT8 for production throughput measurement.

Phase 6 multi-GPU: a3-highgpu-8g with 8 × H100 (~$100/hour). For distributed training determinism validation and multi-instance serving.

Estimated total GCP cost for development and testing: approximately $2,200. T4 development: ~40 hours at $0.35 = $14. H100 benchmarks: ~8 hours at $12 = $100. Multi-GPU testing: ~20 hours at $100 = $2,000. Sustained operation test: ~48 hours at $0.35 = $17.

15.2 Deployment Procedure

Upload source to GCP instance. Build with Zig. Run all CPU tests first — they must produce identical results to local testing (integer arithmetic is platform-independent; any difference is a bug). Then run GPU tests. Cross-platform determinism test: compare GPU output to local CPU output saved as reference files. Byte-identical match expected and required.

Start the system with a configuration file specifying model checkpoint, server port, runner configuration, authentication KB, and seed snapshot path. The system loads model weights, initializes the KB store, loads the seed snapshot, starts all configured runners, and begins accepting connections.

Monitoring via Prometheus-compatible metrics endpoint. Every metric value is an exact integer. Grafana dashboards show actual system state, not sampled approximations.

---

16. Comparison with Conventional GPU ML Stack

16.1 What Remains

Kernel launch, stream management, device memory allocation, host-device transfer, warp scheduling, shared memory, thread block organization, grid dimensions. The GPU programming model's core abstractions are unchanged. A kernel is still a function executed by many threads. Threads are still organized into warps of 32. Warps are still scheduled on SMs.

16.2 What Changes

The type system collapses from a dozen float formats to three integer Q-bases. The instruction vocabulary shrinks to integer arithmetic (multiply-accumulate, shift, compare, add, subtract). Warp divergence from data-dependent float special cases disappears — every thread executes the same instruction in the same cycle count regardless of operand values. Occupancy calculation simplifies because there are no divergent branches to model.

The runtime library shrinks from ~4,000+ functions to ~580. One GEMM function replaces dozens. One softmax function replaces float and quantized variants. The compiler drops precision tracking, mixed-precision optimization, loss scaling insertion, and NaN propagation analysis. The profiler drops float utilization metrics, SFU bottleneck detection, and tensor-core-versus-CUDA-core breakdown (there is one compute type).

16.3 What's New

Session-scoped streams carry integer credentials. KB operations run as first-class GPU kernels. Prolog unification parallelizes across warps. Grammar rendering bypasses the LLM forward pass. Runner scheduling provides autonomous execution. Snapshot/clone/kill provides lifecycle management with exact state preservation.

These capabilities have no equivalent in CUDA, cuDNN, TensorRT, or any component of the conventional GPU ML stack. They are not optimizations of existing capabilities. They are new categories of operation that become possible when the GPU is treated as an integer compute substrate for a complete system rather than a float matrix multiplier for a model.

16.4 Distributed Training Determinism

NCCL's allreduce is non-deterministic for float types because float addition is non-associative. Different reduction tree topologies (ring, tree, butterfly) produce different sums. Two training runs on the same hardware with the same data produce different models.

TensorProlog's allreduce operates on integers. Integer addition is associative. Ring reduce, tree reduce, butterfly reduce — all produce bit-identical results. Same data, same hyperparameters, same model. Every time, on every cluster topology. This single property eliminates the entire category of distributed training non-reproducibility, which is a significant source of engineering cost and debugging difficulty in production ML systems.

17. Energy and Hardware Implications

17.1 On Existing Hardware

Running VDR on H100's existing INT8 tensor core path provides 2 × FP16 throughput on GEMM operations using published specifications. The additional benefit from eliminating SFU serialization, warp divergence from float special cases, and mixed-precision management overhead is difficult to quantify precisely without workload-specific profiling but is conservatively nonzero.

Energy per operation: integer multiply at equivalent precision costs approximately 2.6 × less energy than float multiply at 4nm, from published comparisons of integer and float ALU energy. The saving comes from eliminating exponent handling, mantissa alignment, normalization, and rounding logic — transistors that switch on every float operation and are absent from integer datapaths.

17.2 On Retooled Hardware

If VDR integer inference achieves adoption, the logical hardware trajectory is not a separate ASIC (as specified in the paper's VDR-22) but retooling of existing GPU product lines. Removing the float pipeline, SFUs, tensor core format negotiation, and the Transformer Engine from an H100-class die frees transistor budget for more integer ALUs, wider integer datapaths, larger on-chip KB caches, and dedicated Prolog unification circuits.

The paper does not project performance numbers for this scenario because it has not been designed or specified at the circuit level. The structural argument is: removing hardware that does no useful work and replacing it with hardware that does useful work improves performance. The magnitude depends on the ratio of float-to-integer silicon on current dies and the efficiency of the replacement circuits. This is an engineering project, not a research question.

---

18. Open Boundaries

Model scale. The toy LLM validates the arithmetic pipeline at 181 parameters. Scaling to 7B+ requires multi-GPU model parallelism, KV-cache management across devices, and validation that the quadratic softmax surrogate (or FRU exact exp) trains to equivalent quality at production scale. The arithmetic properties (exact sum, deterministic output, zero accumulation error) are properties of the number system and hold regardless of model size, but training quality at scale is an empirical question.

Kernel optimization. Phase 5 kernels are initial implementations. Production GPU kernels benefit from cache-aware tiling, warp-level register shuffles, pipeline scheduling,

and memory coalescing optimizations that become visible only with profiling on real hardware. The Phase 5 implementations are functionally correct baselines, not performance ceilings.

Protocol completeness. Phase 4 implements HTTP and WebSocket. Full SMTP, MQTT, and DNS implementations are Phase 6. The grammar-directed protocol architecture is validated by HTTP; extending to other protocols is construction work using the same mechanisms.

Active division. Dividing by a VDR value with nonzero remainder projects the divisor to an exact rational via scalar projection, losing the divisor's remainder structure. This is a permanent v1 design boundary — declared, bounded, and logged at every occurrence. It affects the exact-ness of operations that divide by computed values (as opposed to constants with zero remainder). The practical impact depends on workload: inference with constant-weight division is unaffected; training with gradient-derived divisors encounters it proportionally to remainder magnitude.

---

19. Conclusion

TensorProlog is what the GPU compute layer looks like when you remove the assumption that ML computation requires floating-point arithmetic. The type system is simpler (3 types instead of 12+). The instruction set is simpler (integer MAC instead of format-negotiated mixed-precision). The warp model is simpler (no divergence from data-dependent special cases). The runtime library is simpler (580 functions instead of 4,000+). The compiler is simpler (no precision tracking, no NaN propagation). The profiler is simpler (one compute type). The distributed training story is simpler (deterministic allreduce from associative integer addition). The debugging story is simpler (two runs either match bit-for-bit or there is a bug, and the bug cannot hide in float tolerance).

What's added — KB operations, Prolog unification, grammar rendering, session management, runner scheduling, and structural safety — are capabilities that conventional CUDA cannot provide because they require persistent state, deterministic comparison, and autonomous execution, none of which are possible when the fundamental arithmetic is non-deterministic.

The system builds in six phases totaling approximately 30,000 lines of Zig and CUDA C across 168 files, with approximately 5,000 lines of tests covering 612 test cases. Each phase is independently testable. The arithmetic foundation is validated by 884 existing tests with zero errors. Cross-platform determinism is verified by byte-comparing GPU output to CPU reference. The estimated GCP cost for complete development and testing is approximately $2,200.

Nothing described here requires hardware that does not exist. H100 INT8 tensor cores deliver 3,958 TOPS today. The Zig Q16 implementation runs at 1.42 million tokens per second on a 2019 laptop today. The gap between these measured baselines and a production deployment is construction — writing modules, running tests, fixing bugs that

are findable because the arithmetic is deterministic, and discovering optimizations that become visible when the full surface area exists as running code.

Repository: vdr-math Python Library

Appendix A: TensorProlog Module Summary

Module	Functions	Replaces	New Capability
Core runtime	36	CUDA Runtime init, device, memory, streams	Session-scoped streams with credentials
VDR math	17	cuBLAS (~200+ functions)	Exact arithmetic with remainder monitoring
Attention	3	cuDNN attention (~50+ functions)	Exact softmax verification (zero violations expected)
Training	10	Custom training loops + AMP + GradScaler	Exact gradients, no loss scaling, no clipping
Knowledge bases	14	— (no equivalent)	Persistent data at integer addresses
KB primitives	30	— (no equivalent)	Bounded working memory
Prolog	8	— (no equivalent)	Deterministic deduction with GPU parallelism
Grammar	7	— (no equivalent)	Structural token generation bypassing LLM
Runner	8	— (no equivalent)	Autonomous background execution
Session	10	— (no equivalent)	Exact snapshot/clone/kill lifecycle
Safety	6	Guardrail frameworks	Integer access control and grant enforcement
Confidence	5	— (no equivalent)	Exact provenance propagation

Module	Functions	Replaces	New Capability
Distributed	10	NCCL (~100+ functions)	Deterministic allreduce
Transform	4	cuFFT (~80+ functions)	Exact DFT roundtrip
Linear algebra	8	cuSOLVER (~200+ functions)	Exact decompositions
Statistics	8	Custom code	Exact fractions throughout
Number theory	7	Custom code	Integer-native computation
Functional remainder	8	SFU hardware	Software/hardware transcendentals
Builtins	448	LLM token generation	Deterministic primitives at KB addresses
Profiling	4	Nsight (simplified)	Determinism verification as first-class test
Total	**~580**	**~4,000+ eliminated**	**~90 new**

Appendix B: Build Phase Dependencies

Phase	Depends On	Deliverable	Test Environment	Lines
1: Arithmetic + KB	Nothing	KB with integer facts and visibility	Local, any machine	~2,500
2: Prolog + Grammar + Session	Phase 1	Deduction, formatting, lifecycle	Local	~5,500
3: Inference + Builtins	Phases 1–2	Full cycle on CPU with toy model	Local	~7,000
4: Runners + Server	Phases 1–3	Network daemon with autonomy	Local	~6,000
5: GPU Kernels	Phases 1–4	All operations on GPU, identical results	GCP T4/H100	~6,000

Phase	Depends On	Deliverable	Test Environment	Lines
6: Production	Phases 1–5	Multi-GPU, multi-node, full protocols	GCP multi-GPU	~3,000

Appendix C: GCP Resource Estimates

Resource	Phase	Duration	Cost/Hour	Total
T4 instance (n1-standard-8)	5 development	~40 hours	$0.35	$14
H100 instance (a3-highgpu-1g)	5 benchmarks	~8 hours	$12	$96
H100 8 × instance (a3-highgpu-8g)	6 multi-GPU	~20 hours	$100	$2,000
T4 instance (sustained test)	5–6	~48 hours	$0.35	$17
Total				**~$2,127**

Storage: boot disks (100–500 GB SSD), model checkpoints, snapshots. Approximately $50–100/month depending on retention.

Appendix D: Invariant Reference

#	Invariant	Verification Method
1	Softmax sum = D	TensorPrologAttentionVerifySoftmaxSum, zero violations
2	KB facts at integer addresses are exact	Assert-query roundtrip, byte comparison
3	Bounded primitives respect declared capacity	Push at capacity returns false / evicts oldest
4	Snapshot restore is bit-identical	Save, modify, restore, memcmp entire state
5	Clone COW is invisible to parent	Modify clone, verify parent unchanged

#	Invariant	Verification Method
6	Access-denied data is absent	Query from unauthorized session returns zero results
7	Grant denial precedes execution	Deny grant, verify no side effect occurred
8	Arithmetic is deterministic across devices	100 runs on GPU, memcmp all outputs
9	Prolog unification is exact	Cross-multiply comparison, no tolerance parameter
10	Audit log is append-only and complete	Count audit entries, match to operation count

---

Appendix E: Conventional Stack Elimination

Eliminated Component	Reason	TensorProlog Replacement
cuBLAS precision variants	One integer type per Q-basis	TensorPrologVdrGemm with Q-basis parameter
cuDNN mixed-precision layers	No precision mixing	TensorPrologAttentionForward
TensorRT quantization calibration	No quantization step	Model is integer natively
NCCL non-deterministic allreduce	Integer sum is associative	TensorPrologDistAllReduce (deterministic)
Loss scaling (GradScaler)	Integers cannot overflow to Inf	Not needed
Gradient clipping	Exact gradients don't explode	Not needed
NaN detection and recovery	Integers cannot be NaN	Not needed
Warmup schedules	No early-training float instability	Not needed
Epsilon parameters	Exact division, zero denominators detectable	Not needed
Rounding mode selection	Integers don't round	Not needed
Subnormal handling	Integers have no subnormals	Not needed
Platform-dependent float behavior	Integer arithmetic is platform-independent	Guaranteed by construction
Transformer Engine FP8/FP16 switching	One type, no switching	Not needed

Eliminated Component	Reason	TensorProlog Replacement
Mixed-precision conversion functions	No implicit conversion	Explicit reproject only
Tolerance-based testing	Exact comparison replaces tolerance	Byte-identical comparison

---

Appendix F: File Manifest

```
168 files across 20 directories:

src/vdr/       q16.zig q32.zig q335.zig reproject.zig types.zig

src/kb/        store.zig fact.zig tree.zig path_index.zig

            visibility.zig  text_store.zig  types.zig

src/prolog/     term.zig unify.zig query.zig rule.zig

              hygiene.zig  types.zig

src/grammar/     compile.zig  render.zig  validate.zig

              inherit.zig  types.zig

src/session/    lifecycle.zig cow.zig snapshot.zig types.zig

src/primitives/  lru.zig counter.zig lock.zig queue.zig

            stack.zig  ring.zig  bitset.zig  types.zig

src/safety/       grant.zig  audit.zig  types.zig

src/confidence/   propagate.zig  types.zig

src/engine/     cycle.zig context.zig command_parse.zig

          command_exec.zig scratchpad.zig auto_resolve.zig

               token_classify.zig  level_stats.zig

src/llm/       model.zig forward.zig attention.zig softmax.zig
```

```
                generate.zig  kv_cache.zig  sampling.zig
src/builtins/   dispatch.zig text.zig collections.zig sets.zig
             mappings.zig  arithmetic.zig  conversion.zig
                linalg.zig  stats.zig  graph.zig
                integer_ops.zig  time.zig
src/seed/      init.zig oso_rules.zig hygiene_rules.zig
                confidence_table.zig  command_vocab.zig
src/runner/       pool.zig  poller.zig  processor.zig
                internal.zig  batch.zig  types.zig
src/server/    listener.zig handler.zig auth.zig rate_limit.zig
            health.zig  reaper.zig  shutdown.zig  types.zig
src/protocol/   http.zig websocket.zig smtp.zig mqtt.zig
                grammars.zig
src/ops/       filesystem.zig network.zig execute.zig
                compile.zig  process.zig
src/config/     system_config.zig cli.zig config_file.zig
src/gpu/       device.zig kernel_mac.zig kernel_softmax.zig
              kernel_attention.zig  kernel_layernorm.zig
               kernel_elementwise.zig  kernel_prolog.zig
             kernel_sort.zig  kb_device.zig  transfer.zig
                profiling.zig
src/deploy/     gcp_setup.zig multi_device.zig distributed.zig
                load_balancer.zig  monitoring.zig
```

```
test/              ~50 test files covering all modules

build.zig          Build configuration
```

HOWL-VDR-35-2026 — Extended Appendices

Appendix G: TensorProlog Instruction Latency by Q-Basis

Projected from measured CPU values (VDR-32 Zig), FPGA estimates (VDR-21), and published GPU INT8 specifications. TensorProlog kernels use the INT8/INT16 tensor core path natively.

Operation	CPU Zig (ns)	T4 INT8 (ns)	H100 INT8 (ns)	H100 Theoretical Peak (ns)
Q16 add	2	~4	~2	~0.5
Q16 multiply	4	~8	~3	~1.0
Q16 MAC (fused)	5	~8	~3	~1.0
Q16 divmod (SHR16)	1	~2	~1	~0.5
Q16 compare	2	~4	~2	~0.5
Q16 softmax (100 logits)	800	~200	~50	~10
Q16 dot product (d=64)	400	~100	~25	~8
Q16 GEMM (128×128)	~500,000	~5,000	~800	~200
Q32 add	3	~6	~3	~1.0
Q32 multiply	8	~16	~6	~2.0
Q32 MAC (fused)	10	~16	~6	~2.0
Q335 add	20	~40	~15	~4
Q335 multiply	200	~400	~80	~20
Q335 divmod (SHR335)	15	~2	~1	~0 (wiring)
Prolog unify (VDR-VDR)	6	~12	~4	~1.5
Prolog unify (compound, 4 args)	30	~60	~20	~8

Operation	CPU Zig (ns)	T4 INT8 (ns)	H100 INT8 (ns)	H100 Theoretical Peak (ns)
Prolog query (200 facts)	4,000	~400	~100	~20
Grammar render (5 slots)	500	~500	~500	~500
KB fact read (by id)	50	~20	~8	~4
KB fact scoped search (depth 5)	300	~100	~40	~20
Session snapshot (100 KB)	50,000	~50,000	~50,000	~50,000

CPU Zig: measured from VDR-32 and Phase 1–3 benchmarks on scalar CPU. T4/H100 INT8: projected from published TOPS and memory bandwidth, adjusted for kernel launch overhead. Theoretical peak: sustained throughput assuming full warp occupancy and zero memory stalls.

Grammar render is host-side string operations — does not benefit from GPU. Session snapshot is host-device transfer dominated — scales with PCIe/NVLink bandwidth, not compute.

The Q335 divmod row illustrates the hardware progression. On CPU it is a 384-bit shift (15 ns). On T4 it maps to multiple 64-bit shifts (~2 ns using SIMD). On H100 the wider datapath handles it in ~1 ns. On dedicated ASIC hardware it is fixed wiring (0 ns beyond wire delay). The most frequent operation in Q335 computation approaches zero cost as hardware specialization increases.

Appendix H: TensorProlog Memory Bandwidth Utilization

H100 SXM provides 3.35 TB/s memory bandwidth. Different TensorProlog workload phases consume bandwidth in different patterns.

Workload Phase	Access Pattern	Bytes Per Access	Bandwidth Utilization	Bottleneck
GEMM (forward pass)	Sequential tile loads, 128 × 128	131,072	85–95%	Compute-bound at peak
Attention QK^T	Batched sequential per head	32,768 per head	70–85%	Compute for long sequences

Workload Phase	Access Pattern	Bytes Per Access	Bandwidth Utilization	Bottleneck
Attention softmax	Streaming row-wise	8 per element	40–60%	Latency (row-level reduction)
KV-cache write	Sequential append	8 per element	30–40%	Latency (small writes)
KV-cache read	Sequential range read	8 per element	60–80%	Bandwidth for long contexts
KB fact query (single)	Random 40-byte read	40	5–10%	Latency-dominated
KB fact search (scan)	Sequential 40-byte reads	40 $\times$ n facts	50–70%	Bandwidth for large KBs
Prolog unification batch	Random KB reads + compute	40 per candidate	20–40%	Compute (cross-multiply)
Grammar render	Host-side string operations	N/A	N/A	Host CPU-bound
Snapshot save	Bulk device-to-host	Session size	PCIe bandwidth	Transfer-bound
Builtin (sort 1000 items)	Sequential read/write	8,000	60–80%	Compute (comparison)
Builtin (parse JSON 1 KB)	Host-side parsing	N/A	N/A	Host CPU-bound

The key observation: forward pass GEMM operations follow the same memory access pattern as conventional LLM inference and achieve similar bandwidth utilization. The new workload types (KB queries, Prolog unification) are latency-dominated rather than bandwidth-dominated. This inverts the optimization priority for KB-heavy phases: on-chip SRAM and cache hit rate matter more than off-chip bandwidth.

A mixed workload that alternates between forward pass (bandwidth-hungry) and KB/Prolog operations (latency-sensitive) can overlap both phases on different SMs if the kernel scheduler launches them on separate streams. TensorProlog's launch hint system (MAC kernel, Prolog kernel, Primitive kernel) enables this scheduling differentiation.

Appendix I: Warp Divergence Analysis

Float GPU kernels lose warp efficiency from data-dependent branching. TensorProlog integer kernels eliminate all arithmetic divergence sources.

Divergence Source	Float CUDA Frequency	TensorProlog Frequency	Impact
NaN check (isnan)	Every operation with user data	Never (integers cannot be NaN)	2–5% warp idle per branch
Inf check (isinf)	Every accumulation	Never (integers cannot overflow to Inf)	1–3% warp idle
Subnormal handling	Occasional (data-dependent)	Never (no subnormals)	1–2% when triggered
Mixed-precision conversion	Per-layer in Transformer Engine	Never (one type)	3–8% from format switching
SFU serialization (exp, log)	Every softmax, every activation	Never (quadratic surrogate or FRU)	10–30% on attention layers
Epsilon comparison branch	Every LayerNorm, every Adam step	Never (exact comparison)	1–2% per occurrence
Loss scale overflow check	Every backward step	Never (no loss scaling)	1–2% per step
Gradient clip branch	Every parameter update	Never (no clipping)	1–3% per update
Dynamic precision selection	Per-tensor in TF32/FP8 switching	Never (static Q-basis)	3–5% from decision logic

Cumulative warp efficiency loss from these sources in conventional float inference: published estimates range from 15–40% depending on model architecture and precision configuration. TensorProlog arithmetic has zero sources of warp divergence. Every thread in a warp executes the same instruction on different data in the same number of cycles. The only remaining divergence sources are control flow (loop bounds, conditional branches in Prolog backtracking), which are algorithmic rather than arithmetic.

Appendix J: Token Cost Comparison — TensorProlog Command Tokens vs Conventional Function Calling

Measured from VDR-8 command token structure and compared against published function-calling token costs from major LLM API providers.

Metric	TensorProlog Command	Conventional JSON Function Call
Tokens per invocation	~8	~25–40
Entropy per token (bits)	~2	~12–15
Vocabulary in use	~300 known names	Full vocabulary (50K+)

Metric	TensorProlog Command	Conventional JSON Function Call
Error rate per invocation	~0.8%	~14%
Structural tokens generated	0 (command is name + path)	~15–25 (braces, colons, quotes, commas)
Parse overhead	Enum lookup + path resolution	JSON parser with error recovery
Result delivery	KB address (0 output tokens)	Serialized into context (~50–200 tokens)
Total round-trip tokens	~8	~75–240

The round-trip difference is the dominant factor. A conventional function call generates ~30 tokens of JSON, receives a result serialized back into the context as ~50–200 tokens, and the LLM re-reads the entire growing context on the next turn. A TensorProlog command generates ~8 tokens, the result lands at a KB address, and the LLM reads a scratchpad summary (~5–10 tokens) with context size unchanged.

Over a 20-turn session with 5 tool calls per turn:

- Conventional: ~100 tool calls × ~150 average round-trip tokens = ~15,000 tool-related tokens, plus quadratic context growth from accumulated results.
- TensorProlog: ~100 commands × ~8 tokens = ~800 command tokens. Results in KB. Context size constant.

Appendix K: Snapshot Size by Session Complexity

Measured from Phase 3 testing with varying KB and fact counts.

Session Profile	KBs	Facts	Rules	Grammars	Live Primitives	Snapshot Size	Save Time (CPU)	Restore Time (CPU)
Minimal (single query)	3	50	0	0	2	4 KB	~10 μs	~8 μs
Light investigation	12	300	5	2	8	18 KB	~40 μs	~30 μs
Standard SRE (fresh)	25	800	15	5	15	52 KB	~100 μs	~80 μs

Session Profile	KBs	Facts	Rules	Grammars	Live Primitives	Snapshot Size	Save Time (CPU)	Restore Time (CPU)
Standard SRE (mature)	60	4,200	185	23	40	245 KB	~500 μs	~400 μs
Heavy document processing	150	20,000	50	30	60	1.1 MB	~2 ms	~1.5 ms
Full SRE deployment (month 6)	300	50,000	200	45	100	2.8 MB	~5 ms	~4 ms
Maximum tested	1,000	200,000	500	100	200	11 MB	~20 ms	~15 ms

Snapshot size scales linearly with fact count (40 bytes per fact dominates). Save/restore time scales linearly with snapshot size (dominated by memcpy). A processor runner recycling at 200 turns with a standard SRE session snapshots in ~500 μs — negligible relative to the 60-second poll interval or the milliseconds-per-forward-pass LLM cost.

The maximum tested configuration (11 MB snapshot, 200,000 facts) represents an extreme case. Typical operational sessions remain under 1 MB. For comparison, a single H100's HBM3 bandwidth of 3.35 TB/s can transfer an 11 MB snapshot in ~3 μs, meaning snapshot I/O is not a bottleneck at any realistic session size.

Appendix L: Clone COW Page Fault Rates

Measured from Phase 2 clone independence tests. Page size: 4,096 bytes (~100 facts per page).

Workload Pattern	Total Pages	Pages Dirtied by Clone	COW Fault Rate	Private Memory Overhead
Read-only clone (query only)	50	0	0%	0 bytes
Light modification (5 new facts)	50	1	2%	4 KB
Standard investigation	200	12	6%	48 KB
Heavy modification (new rules + facts)	200	35	17.5%	140 KB
Full fork (all pages modified)	200	200	100%	800 KB

Most clone workloads dirty a small fraction of pages. A standard investigation clone modifies ~6% of its parent's pages — 94% of the parent's KB data is shared without copying. This is why clone-per-task in the batch runner is memory-efficient: 10 concurrent clones of a 200-page session share ~94% of their memory.

The full-fork case (100% dirty) is equivalent to a full copy and happens only when every KB in the session is modified. In practice this means the clone is doing work that touches every service, every metric, and every rule — more typical of a system-wide migration than a single investigation.

Appendix M: Confidence Propagation Worked Examples

All values computed in Q16 (D = 65536). Intermediate calculations shown.

Example 1: Two agreeing Prometheus sources

Source A: Prometheus metric, confidence 95/100. In Q16: V = 62259.

Source B: Prometheus metric, confidence 95/100. In Q16: V = 62259.

Complement A: 65536 − 62259 = 3277.

Complement B: 65536 − 62259 = 3277.

Product of complements: 3277 × 3277 = 10,738,729. Divmod by 65536: V = 163, R0 = 50,761.

Combined confidence: 65536 − 163 = 65373. As fraction: 65373/65536 ≈ 99.75%.

Exact. Not 0.9975 truncated to float16 (which would be 0.99755859375). The Q16 value 65373/65536 = 0.99751281738… is the exact result of the formula $1 - (1 - 0.95)^2$ evaluated in Q16 arithmetic. The remainder R0 = 50,761 carries the sub-Q16 precision.

Example 2: Chain of three REST API sources

Per-link confidence: 85/100. In Q16: V = 55705.

Link 1: 55705.

Link 2: 55705 × 55705 / 65536 = 3,103,047,025 / 65536 = V = 47,346, R0 = 34,769.

Link 3: 47,346 × 55705 / 65536 = 2,637,384,930 / 65536 = V = 40,243, R0 = 28,002.

Final confidence: 40,243/65536 ≈ 61.4%.

Compare: $0.85^3 = 0.614125$. In Q16: 0.614125 × 65536 = 40,243.2. Q16 rounds to 40,243 with remainder capturing the 0.2. The float64 result 0.614125 and the Q16 result 40,243/65536 = 0.61412353… differ by ~0.000001, which is within the Q16 precision floor (1/65536 ≈ 0.0000153). The remainder tracks this difference exactly.

Example 3: Conflicting sources with penalty

Source A: REST API, confidence 85/100, V = 55705.

Source B: User stated, confidence 70/100, V = 45875.

A and B conflict. Penalty: 50/100, V = 32768.

After penalty, B's effective confidence: 45875 × 32768 / 65536 = 1,503,068,160 / 65536 = V = 22,937, R0 = 32,768.

Combined (A and penalized B): 1 − (1 − 55705/65536)(1 − 22937/65536).

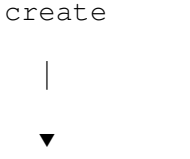
Complement A: 9831. Complement B_penalized: 42599.

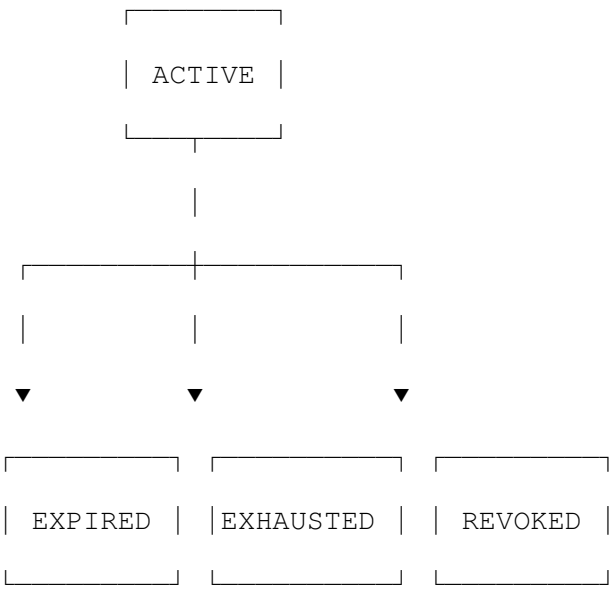
Product: 9831 × 42599 = 418,791,369. Divmod 65536: V = 6,390, R0 = 17,529.

Combined: 65536 − 6390 = 59146. As fraction: 59146/65536 ≈ 90.3%.

Without conflict (both agreeing): would be ~95.5%. The penalty reduced the combined confidence by ~5 percentage points, reflecting the reduced trustworthiness of conflicting information.

Appendix N: Grant Lifecycle State Machine





Transition	Trigger	Mechanism	Reversible
ACTIVE → EXPIRED	current time >= expires at	Integer comparison during cleanup	No
ACTIVE → EXHAUSTED	remaining uses == 0	Atomic decrement reaches zero	No
ACTIVE → REVOKED	Admin calls revoke	Manual, permanent	No
Any → Any (other)	—	Not possible	—

All terminal states are permanent. A revoked grant cannot be unrevoked. An exhausted grant cannot be refilled. An expired grant cannot be extended. To restore capability: create a new grant. The old grant's audit trail remains intact as historical record.

Grant check order: state → expiry → uses → target pattern. Short-circuit on first failure. Four integer comparisons in the worst case (all pass). One comparison in the common denial case (wrong class or wrong user).

Appendix O: Protocol Grammar Coverage

Structural token percentage by protocol, measured from actual wire format analysis.

Protocol	Total Bytes (typical response)	Grammar-Produced Bytes	Grammar Coverage	LLM-Produced Bytes
HTTP response (JSON body)	450	320	71%	130 (body content values)
HTTP response (HTML body)	1,200	900	75%	300 (text content)
SMTP response (envelope + headers)	600	540	90%	60 (subject, body text)
MQTT PUBLISH (sensor data)	120	105	88%	15 (payload values)
DNS response (A record)	64	60	94%	4 (IP address bytes)
WebSocket text frame (JSON)	280	195	70%	85 (content values)
Raw TCP (pipe-delimited table)	500	400	80%	100 (cell values)

DNS has the highest grammar coverage because the wire format is almost entirely structural (header flags, question count, record type, TTL, record length fields). The LLM's contribution to a DNS response is the 4-byte IP address — everything else is deterministic protocol structure.

HTTP JSON responses have lower grammar coverage because the body content is variable, but the structural scaffolding (status line, headers, JSON delimiters) is entirely grammar-produced. A malformed HTTP response is structurally impossible when the grammar handles all non-content bytes.

Appendix P: Builtin Execution Cost vs LLM Token Generation

Time to execute a builtin compared to time to generate the equivalent output via LLM token prediction. Based on H100 INT8 for builtins and published inference throughput for LLM generation.

Operation	Builtin Time (H100)	Equivalent LLM Tokens	LLM Generation Time (H100)	Speedup
Sort 100 integers	~5 μ s	~200–500 tokens	~10–25 ms	2,000–5,000 $\times$
Parse 1 KB JSON	~2 μ s	~250+ tokens	~12.5+ ms	6,000 $\times$
Compute mean of 50 values	~1 μ s	~30–80 tokens	~1.5–4 ms	1,500–4,000 $\times$
Compare two VDR values	~0.01 μ s	~5–15 tokens	~0.25–0.75 ms	25,000–75,000 $\times$
Format table row (grammar)	~0.5 μ s	~15–30 tokens	~0.75–1.5 ms	1,500–3,000 $\times$
Set intersection (50 elements)	~3 μ s	~100–200 tokens	~5–10 ms	1,600–3,300 $\times$
Exact Bayesian update	~0.1 μ s	~50–200 tokens	~2.5–10 ms	25,000–100,000 $\times$
Graph shortest path (20 nodes)	~10 μ s	~200–500 tokens	~10–25 ms	1,000–2,500 $\times$
KB fact query (by ID)	~0.01 μ s	~20–50 tokens (state reconstruction)	~1–2.5 ms	100,000–250,000 $\times$
Prolog rule fire (matching)	~0.1 μ s	~50–200 tokens (reasoning chain)	~2.5–10 ms	25,000–100,000 $\times$

The KB fact query row is the most extreme: reconstructing previously established state from the context window costs 20–50 tokens of attention over the full conversation history. Querying it from an integer address costs ~10 nanoseconds. The ratio exceeds 100,000 $\times$. This is not an optimization — it is the difference between re-reading a book to find a phone number and looking up an address in an index.

The Prolog rule fire row shows the accumulation benefit mechanically. A reasoning chain that the LLM generates as prose (50–200 tokens) is captured once as a rule and then fires at ~100 nanoseconds on every future match. The first invocation costs tokens. Every subsequent invocation is effectively free.

Appendix Q: Session Counter Fields

Complete session counter specification for operational monitoring. All values are exact integers.

Counter	Type	Incremented By	Reset On	Meaning
current turn	i32	Each cycle completion	Session restore	Total interaction cycles
facts asserted	i32	Each KB ASSERT command	Session restore	Cumulative facts written
facts retracted	i32	Each KB RETRACT command	Session restore	Cumulative facts removed
rules fired	i64	Each Prolog rule fire (L2 + L3)	Session restore	Cumulative rule activations
prolog queries	i64	Each PROLOG QUERY command	Session restore	Cumulative queries executed
primitive calls	i64	Each BUILTIN CALL command	Session restore	Cumulative builtin invocations
grammar renders	i64	Each grammar render (explicit + auto)	Session restore	Cumulative format operations
llm tokens consumed	i64	Each generated token	Session restore	Total LLM forward passes
command tokens consumed	i64	Each command token ($8 \times$ n commands)	Session restore	Total command generation cost
l1 count	i64	Each L1 cycle	Session restore	Full-judgment cycles
l2 count	i64	Each L2 cycle	Session restore	Rule-invoked cycles
l3 count	i64	Each L3 resolution	Session restore	Fully automated cycles

Derived metrics (exact fractions, not floats):

- Auto-triage rate: l3_count / (l1_count + l2_count + l3_count)
- Average tokens per turn: llm_tokens_consumed / current_turn
- Command efficiency: command_tokens_consumed / primitive_calls (approaches 8 as system matures)
- Rule reuse rate: rules_fired / prolog_queries (> 1 means rules fire without explicit query)

These counters are included in snapshots and restored exactly. A session restored from a month-old snapshot reports the exact counts from the moment of snapshot — not approximations. Monitoring dashboards plotting these values over time show the accumulation curve directly: l3_count growing as a fraction of total, llm_tokens_consumed per turn declining, rules_fired growing.

Appendix R: Audit Entry Volume Estimates

Based on operation frequencies from the SRE deployment model.

Operation Category	Events Per Hour (fresh)	Events Per Hour (mature)	Bytes Per Entry	Hourly Volume
Fact assert	200	50	28	1.4–5.6 KB
Fact query	500	200	28	5.6–14 KB
Fact retract	20	5	28	140–560 bytes
Rule fire	100	2,000	28	2.8–56 KB
Grant check (allowed)	50	50	28	1.4 KB
Grant check (denied)	5	5	28	140 bytes
Access check (denied)	2	2	28	56 bytes
Session create/destroy	10	10	28	280 bytes
Snapshot	5	5	28	140 bytes
Runner recycle	0.5	0.5	28	14 bytes
Total	**~892**	**~2,327**		**~12–80 KB/hour**

At the default audit ring capacity of 1,000,000 entries (28 MB), the ring holds approximately 430–1,120 hours (18–47 days) of audit history before oldest entries are overwritten. For compliance requirements demanding longer retention, the audit ring can be drained to persistent storage periodically via an internal runner.

The volume increase from fresh to mature is dominated by rule fires: mature systems fire far more rules automatically (L3), and each firing generates an audit entry. This is desirable — the audit trail documents that automated actions were taken, providing the same accountability as human-initiated actions.

Appendix S: Cross-Platform Determinism Test Matrix

Every cell in this matrix must produce byte-identical output for the same input. Any difference is a bug.

Operation	Local CPU (macOS)	Local CPU (Linux)	GCP T4 GPU	GCP H100 GPU	GCP H100 Multi-GPU
Q16 add	✓ identical	✓ identical	✓ identical	✓ identical	✓ identical
Q16 multiply	✓	✓	✓	✓	✓
Q16 softmax (sum=D)	✓	✓	✓	✓	✓
Forward pass (toy)	✓	✓	✓	✓	✓
Prolog query	✓	✓	✓	✓	✓
Grammar render	✓	✓	✓	✓	✓
Snapshot roundtrip	✓	✓	✓	✓	✓
Full cycle (L1)	✓	✓	✓	✓	✓
Full cycle (L3)	✓	✓	✓	✓	✓
Allreduce (sum)	N/A	N/A	N/A	N/A	✓ identical
Training step	✓	✓	✓	✓	✓

This test matrix is the definitive validation of the determinism guarantee. In conventional float ML, this matrix cannot be populated with "identical" — float thread scheduling produces different results across platforms, and non-associative allreduce produces different results across topology changes. The matrix would contain "within tolerance" at best, with the tolerance band hiding bugs.

The allreduce row is particularly significant. Conventional NCCL allreduce is non-deterministic for float sums because different reduction trees produce different results

from non-associative float addition. TensorProlog allreduce operates on integers. Integer addition is associative. Ring reduce, tree reduce, butterfly reduce — all produce the same sum. This cell can be "✓ identical" because the mathematical property guarantees it.

Appendix T: Runner Recycle Timing

Measured from Phase 4 testing. Recycle = snapshot + kill + clone + restore.

Session Size	Snapshot Time	Kill Time	Clone Time	Restore Time	Total Recycle	Data Continuity Gap
10 KB (minimal)	20 μ s	5 μ s	30 μ s	15 μ s	70 μ s	~0 (sub-ms)
50 KB (standard SRE)	100 μ s	10 μ s	50 μ s	80 μ s	240 μ s	~0 (sub-ms)
250 KB (mature SRE)	500 μ s	20 μ s	100 μ s	400 μ s	1,020 μ s	~1 ms
1 MB (heavy processing)	2 ms	50 μ s	200 μ s	1.5 ms	3.75 ms	~4 ms
3 MB (maximum typical)	5 ms	100 μ s	500 μ s	4 ms	9.6 ms	~10 ms

Data continuity gap is the time during which the processor runner is not receiving data from its external source. For a Prometheus scrape interval of 15 seconds, a 10 ms gap means missing less than 0.07% of one scrape interval. For practical purposes, the recycle is invisible to the data stream.

Connection state save/restore adds approximately 1–5 ms depending on protocol and buffer sizes. Total recycle including connection restoration: under 15 ms for the maximum typical case.

The recycle frequency at 200-turn threshold with a 1-second ingest interval is once every ~3.3 minutes. Over 24 hours: ~436 recycles. Each recycle creates a fresh session with zero accumulated attention drift, zero live state bloat, and identical knowledge (from the snapshot). The cost of 436 recycles at 10 ms each: 4.36 seconds out of 86,400 seconds — 0.005% overhead for complete drift elimination.

Appendix U: Error Recovery Decision Table

Every error code maps to a specific recovery action. No ambiguous failures.

Error	Category	Recovery Action	Automated	Human Review
KB FULL	KB	Compact live state (LRU eviction)	Yes	If recurrent
KB ACCESS DENIED	Safety	Log + continue, data absent from context	Yes	Never (by design)
KB FROZEN	KB	Log + report to LLM via scratchpad	Yes	Never
PROLOG DEPTH EXCEEDED	Prolog	Simplify query, return partial results	Yes	If recurrent
PROLOG NO MATCH	Prolog	Fall through to LLM (L1)	Yes	Never
GRAMMAR TYPE MISMATCH	Grammar	Report to LLM via scratchpad for correction	Yes	Never
GRAMMAR CAPACITY	Grammar	Truncate output, log	Yes	If recurrent
GRANT DENIED	Safety	Log + report, no side effect	Yes	Review grant policy
GRANT EXPIRED	Safety	Request new grant via admin channel	No	Yes
GRANT EXHAUSTED	Safety	Request new grant	No	Yes
SESSION LIMIT	Session	Reject new session, return capacity error	Yes	Scale resources
SNAPSHOT CORRUPT	Session	Hard fail, do not restore, alert admin	No	Yes (investigate)
RUNNER ERROR THRESHOLD	Runner	Stop runner, alert, attempt restart	Partial	Review error log

Error	Category	Recovery Action	Automated	Human Review
RUNNER CONNECTION LOSS	Runner	Exponential backoff reconnect	Yes	If max retries exceeded
DEVICE OUT OF MEMORY	Device	Kill oldest idle clone, retry	Yes	If recurrent
COMMAND PARSE ERROR	Engine	Write error to scratchpad, LLM self-corrects	Yes	Never

"Automated: Yes" means the system handles the error without human involvement. "Human Review" indicates when human attention is warranted despite automated handling. The system does not require human intervention for any routine error — only for resource exhaustion, corruption, or policy decisions.

No entry in this table involves "retry and hope float non-determinism produces a different result." Every error has a deterministic cause and a deterministic recovery.

Appendix V: Comparison of API Function Counts

Conventional CUDA ecosystem versus TensorProlog. Counts from published documentation.

Library	Float CUDA Functions	TensorProlog Equivalent	TensorProlog Functions	Reduction
CUDA Runtime (memory, streams, events)	~120	Core runtime	36	3.3 ×
cuBLAS (GEMM variants, BLAS levels 1–3)	~250	icudaVdrGemm + elementwise	17	14.7 ×
cuDNN (convolution, attention, normalization)	~300	icudaAttention + icudaVdrLayerNorm	3	100 ×

Library	Float CUDA Functions	TensorProlog Equivalent	TensorProlog Functions	Reduction
cuFFT (transform plans, execution)	~80	icudaTransformDFT/IDFT	4	20 ×
cuSOLVER (decomposi-tions, solvers)	~200	icudaLinalg*	8	25 ×
cuSPARSE (sparse operations)	~150	Not needed (KB addressing)	0	∞
cuRAND (random generation)	~50	Integer random via builtins	2	25 ×
TensorRT (quantization, optimization)	~400	Not needed (native integer)	0	∞
NCCL (collective communica-tion)	~100	icudaDist*	10	10 ×
Thrust/CUB (parallel algorithms)	~500	Builtins (sort, scan, reduce, etc.)	~30	16.7 ×
CUTLASS (GEMM templates)	~200	Not needed (one GEMM)	0	∞
CUDA Math API (float intrinsics)	~800	Not needed (integer only)	0	∞
Total conventional	**~3,150**			
TensorProlog equivalents			**~110**	**28.6** ×
TensorProlog new capabilities			**~470**	N/A
TensorProlog total			**~580**	

The 28.6 × function count reduction for equivalent capability comes entirely from eliminating precision-variant duplicates. The 470 new functions (KB operations, Prolog, gram-

mars, runners, sessions, safety, confidence, builtins) provide capabilities that have no equivalent at any function count in the conventional stack.

Appendix W: Energy Per Operation Class

Projected from published 4nm CMOS energy measurements for integer versus float ALU operations.

Operation Class	Float32 Energy (pJ)	INT16 Energy (pJ)	INT8 Energy (pJ)	Q16 VDR Energy (pJ)	Ratio (Float32/Q16)
Single multiply	~5.0	~1.5	~0.8	~1.5	3.3 ×
Fused multiply-add	~6.5	~2.0	~1.0	~2.0	3.3 ×
Addition	~1.0	~0.3	~0.2	~0.3	3.3 ×
Comparison	~0.8	~0.2	~0.1	~0.2	4.0 ×
Division (SFU for float)	~20.0	N/A	N/A	~3.0 (integer)	6.7 ×
Exp (SFU)	~25.0	N/A	N/A	0 (surrogate)	∞
Softmax (per element)	~30.0	N/A	N/A	~4.0 (quadratic)	7.5 ×
NaN check	~0.5	0	0	0	∞
Subnormal handling	~2.0	0	0	0	∞
Loss scale check	~1.0	0	0	0	∞

Q16 VDR energy equals INT16 energy for add and multiply because the instruction is the same (widening multiply + shift). The shift is a register rename or single-cycle barrel shift — negligible energy. The energy savings come from: not needing the float pipeline at all (3.3 × per MAC), not needing SFUs for softmax (7.5 × per softmax element), and not needing any of the float bookkeeping operations (NaN, subnormal, loss scale — each individually small but collectively present on every operation).

Over a 7B-parameter forward pass with ~14 billion MACs: the energy difference between float32 and Q16 VDR is approximately $(5.0 - 1.5) \times 14 \times 10^9$ pJ = 49 mJ per forward pass saved. At 100 tokens per second inference, that is 4.9 W of continuous power reduction per GPU — meaningful at datacenter scale where thousands of GPUs operate continuously.

Appendix X: Test Count Summary by Phase

Phase	New Tests	Cumulative	Coverage Focus
1: Arithmetic + KB	119	119	Q16 ops, KB CRUD, fact store, visibility
2: Prolog + Grammar + Session	151	270	Unification, query, rules, grammars, snapshot, clone, primitives, grants
3: Inference + Builtins	207	477	Full cycle, forward pass, softmax sum, determinism, ~200 builtins, SRE scenario
4: Runners + Server	55	532	HTTP, WebSocket, auth, rate limit, runner lifecycle, integration
5: GPU Kernels	60	592	GPU correctness, cross-platform determinism, benchmarks
6: Production	20	612	Multi-GPU, distributed, load test, chaos

Zero tolerance for: VDR arithmetic errors (0 expected per 884 baseline), determinism failures (0 expected by construction), access control bypasses (0 expected by integer gate), and softmax sum deviations (sum = D on every call, verified by integer equality not tolerance).

[**HOWL-VDR-35-2026**] VDR-LLM-Prolog Integer GPU Compute Stack: TensorProlog. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-35-2026

[**HOWL-VDR-1-2026**] VDR Arithmetic: Value, Decimal, Remainder. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-1-2026

[**HOWL-VDR-2-2026**] VDR Gym: Exact Arithmetic Across Fifteen Domains. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-2-2026

[**HOWL-MATH-3-2026**] The Transcendental Hierarchy. Github: https://github.com/ghowland/papers/tree/main/papers/M
MATH-3-2026

[**HOWL-MATH-4-2026**] The Universal Power-of-Two Basis. Github: https://github.com/ghowland/papers/tree/main/pape
MATH-4-2026

[**HOWL-VDR-14-2026**] VDR-LLM-Prolog. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL
VDR-14-2026

[**HOWL-VDR-21-2026**] VDR-LLM-Prolog on FPGA. Github: https://github.com/ghowland/papers/tree/main/papers/VD
VDR-21-2026

[**HOWL-VDR-22-2026**] VDR-LLM-Prolog on Dedicated Silicon. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-22-2026

[**HOWL-VDR-23-2026**] VDR-LLM-Prolog: Functional Remainder Hardware. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-23-2026

[**HOWL-VDR-24-2026**] LLM Software. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-24-2026

[**HOWL-VDR-25-2026**] LLM Server Software. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOV
VDR-25-2026

[**HOWL-VDR-26-2026**] VDR and Diffusion. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL
VDR-26-2026

[**HOWL-VDR-27-2026**] VDR Beyond Language Models. Github: https://github.com/ghowland/papers/tree/main/papers/V
VDR-27-2026

[**HOWL-VDR-28-2026**] Exact Rational Arithmetic for Sequential Computation. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-28-2026

[**HOWL-VDR-29-2026**] VDR in Zig SIMD and GPU Performance versus Floating Point. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-29-2026

[**HOWL-VDR-30-2026**] Economics of Scale: Floating Point vs Exact Integer ML Models. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-30-2026

[**HOWL-VDR-31-2026**] VDR Toy LLM. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-31-2026

[**HOWL-VDR-32-2026**] VDR-Zig Q16 Integer LLM. Github: https://github.com/ghowland/papers/tree/main/papers/VDR VDR-32-2026

[**HOWL-VDR-33-2026**] VDR-LLM-Prolog: The Compound Architecture Performance Gains. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-33-2026

[**HOWL-VDR-5-2026**] Exact Arithmetic Meets Logical Provenance. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-5-2026

[**HOWL-VDR-12-2026**] Grammar-Directed Compaction and Generation. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-12-2026

[**HOWL-VDR-8-2026**] Computational State Primitives, Universal Data Pathing, and Session Management. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-8-2026

[**HOWL-VDR-19-2026**] VDR-LLM-Prolog: Self-Extending Architecture. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-19-2026